AMVISH 2.O

[A Self-Aware Robot]

Team Members: -

- **Aman Vishwas**
- **Ujjwal Kant**
- **Aditya Kumar**
- **Ekansh Suman**

DECEMBER 26, 2025

[IMAGINATION INVENTORS]

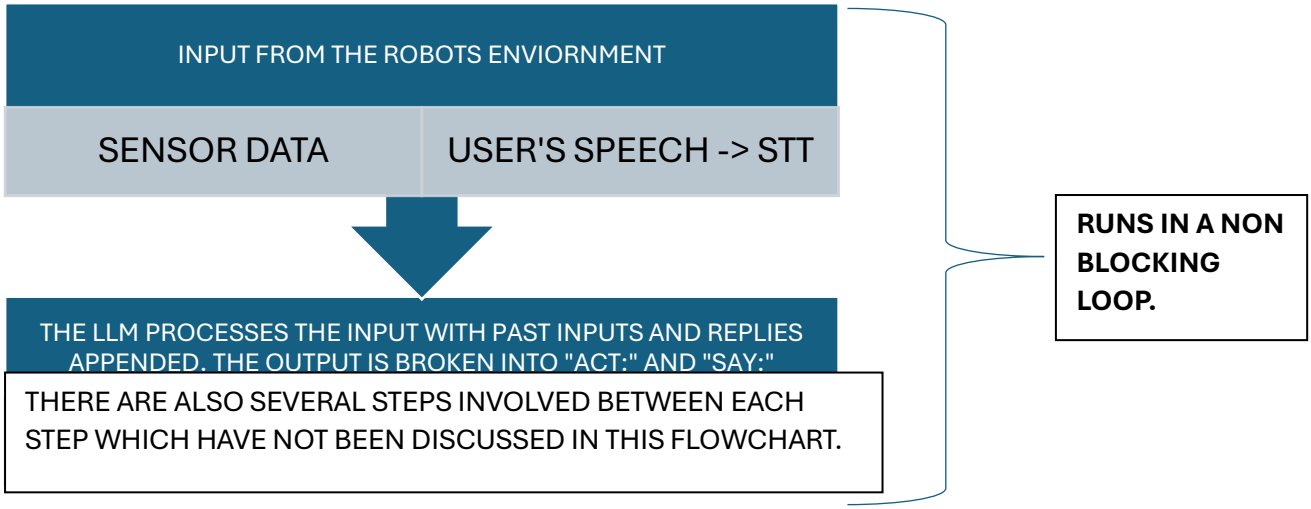
AmViSH_2.O

A Study on Loop-Based Integration of a Large Language Model with Real-Time Robotic Sensory Systems

Abstract

This project investigates the behaviour of a Large Language Model when placed inside a continuous perception–action loop of a physical robotic system. Unlike traditional robotics architectures that rely on deterministic control logic, this system delegates high-level decision-making to an external Large Language Model while maintaining deterministic low-level sensing and actuation. The objective of the study is not to claim learning or intelligence, but to evaluate system stability, responsiveness, and failure modes when probabilistic language models are exposed to real-world sensor noise, identity ambiguity, and dynamic environments. The work focuses on architectural design, runtime error mitigation, and adaptive preprocessing techniques required to make such a system usable outside controlled laboratory conditions.

This can be easily understood with the help of the following flowchart:



Introduction

Robotic systems traditionally operate using **predefined rules** such as **finite state machines (**THIS PROJECT ALSO USES IT, BUT NOT FOR DECISION MAKING**)**, behaviour trees, or hard-coded conditional logic. While these approaches provide predictability, they require **explicit enumeration of all expected situations**. Recent availability of **Large Language Models** introduces the **possibility of using probabilistic inference instead of explicit rules.** However, most existing LLM applications operate in turn-

based, text-only environments. Their behaviour under continuous sensory input and real-world noise is poorly documented. **This project addresses the question of what observable properties and limitations arise when an LLM-driven control layer is embedded in a continuous real-world robotic loop.**

Scope and Non-Claims

This project **does not** claim learning, self-improvement, internal world modelling, cognitive reasoning, or autonomous goal generation. The Large Language Model does not update its weights, does not retain long-term state autonomously, and does not perform sensor fusion independently. Any intelligent-seeming behaviour arises from structured input handling and constrained output interpretation, not from internal understanding.

System Architecture

The system is divided into **deterministic and probabilistic components**. Deterministic components include audio signal acquisition, adaptive energy-based speech detection, vision-based face presence detection, network discovery, communication handling, and hardware actuation interfaces. The probabilistic component is the Large Language Model, which interprets transcribed speech and produces action suggestions and spoken responses. **The LLM never directly accesses raw sensor data**. All inputs are pre-processed and filtered before being converted to text.

Continuous Loop Design

The system operates as an infinite loop. Audio is recorded in fixed-duration chunks, transcribed into text, passed to the LLM, and the resulting action and speech outputs are executed. The loop is non-blocking and tolerant to missing or invalid inputs. This structure exposes the LLM to repeated environmental interaction, timing variability, and non-deterministic input intervals, which is fundamentally different from turn-based chatbot operation.

HERE'S A SNIPPET OF OUR CODE:

```
while True:
    audio = record_chunk()
    text = transcribe(audio)

    if not text:
        continue

    print(f"👂 Heard: {text}")

    response = llm_process(text)

    act = response.get("ACT", "IDLE")
    say = response.get("SAY", "null")

    # ⚙️ Handle robot action
    handle_act(act)

    # 🗣️ Speak only if valid
    if say and say.lower() != "null":
        speak(say)
        time.sleep(0.4)  # natural pacing
```

Adaptive Audio Thresholding

Fixed speech thresholds fail in real environments due to changing background noise, crowd sounds, and echo. To address this, the system maintains a rolling estimate of ambient noise energy using exponential smoothing. Speech is detected only when the signal energy exceeds the learned ambient level by a configurable margin. This approach adapts continuously and reduces false positives without relying on absolute thresholds.

```
# Adaptive noise learning
ALPHA = 0.1                    # smoothing (0.05–0.2)
MARGIN = 1.20                  # speech must be louder than noise × margin

# Speech duration limits
MIN_SPEECH_DURATION = 0.2
MAX_SPEECH_DURATION = 8.0
```

Speech Recognition Constraints

Speech recognition is constrained using duration limits, language filtering, and rejection of low-confidence outputs. Transcriptions are accepted only if the detected language matches predefined languages. This prevents unintended command execution

```
def record_chunk():
    # Initialize ambient level
    if ambient_energy_avg is None:
        ambient_energy_avg = energy
        print(f"[INIT] Ambient set to {ambient_energy_avg:.4f}")
        return None

    # Define thresholds
    noise_learning_threshold = ambient_energy_avg * 0.8
    speech_threshold = ambient_energy_avg * MARGIN

    # 🟢 NOISE LEARNING MODE
    if energy < noise_learning_threshold:
        ambient_energy_avg = (
            (1 - ALPHA) * ambient_energy_avg +
            ALPHA * energy
        )
        print(f"[NOISE] energy={energy:.4f} ambient={ambient_energy_avg:.4f}")
        return None

    # 🔴 SPEECH CANDIDATE
    if energy > speech_threshold:
        print(
            f"[SPEECH] energy={energy:.4f} "
            f"ambient={ambient_energy_avg:.4f} "
            f"threshold={speech_threshold:.4f}"
        )
        return audio

    # In-between → ignore
    print(f"[IGNORE] energy={energy:.4f}")
    return None
```

LLM Control Constraints

The **LLM is strictly sandboxed**. It is allowed to output only predefined action. Any output outside the allowed action set is automatically replaced with a safe idle state. All LLM outputs are parsed, validated, and sanitized before execution. This prevents unsafe behaviour, prompt injection effects, and unintended control commands.

```
# ──────────────────────────────────────────────
# 🧱 STRICT ACT VOCABULARY
# ──────────────────────────────────────────────
ALLOWED_ACTS = {
    "MOVE_FORWARD",
    "MOVE_BACKWARD",
    "TURN_LEFT",
    "TURN_RIGHT",
    "STOP_ALL",
    "WAVE_HAND",
    "IDLE",
    "NULL"
}
```

Dynamic Temperature Control

The response temperature of the LLM is adjusted based on simple keyword-based intent classification. Commands related to motion or control use a low temperature to improve determinism. Creative or conversational inputs use higher temperatures. This reduces hallucinations during control tasks while preserving conversational flexibility.

```
# ──────────────────────────────────────────────
#  DYNAMIC TEMPERATURE SELECTOR
# ──────────────────────────────────────────────
def choose_temperature(user_text: str) -> float:
    text = user_text.lower()

    command_keywords = [
        "move", "go", "forward", "back",
        "turn", "left", "right", "stop", "wave"
    ]

    creative_keywords = [
        "story", "joke", "imagine", "funny", "cool", "dream"
    ]

    if any(k in text for k in command_keywords):
        return 0.25   #  strict robot mode

    if any(k in text for k in creative_keywords):
        return 0.7    #  creative mode

    return 1      #  normal conversation
```

Fault Tolerance

Network discovery of the ESP32 device is performed dynamically using UDP broadcast rather than hardcoded IP addresses. Vision and network failures are isolated so that a failure in one subsystem does not crash the entire robot. Exceptions in non-critical threads are caught and ignored to preserve system continuity.

reverse_discovery.py > discover_esp32

```python
# reverse_discovery.py
import socket, time

UDP_PORT = 4210
PING_MSG = b"AMVISH_PING"
PONG_MSG = b"AMVISH_PONG"

def discover_esp32(timeout=10):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    sock.bind(("0.0.0.0", 0))
    sock.settimeout(1)

    print("📡 Searching for ESP32-CAM...")
    start = time.time()

    while time.time() - start < timeout:
        sock.sendto(PING_MSG, ("255.255.255.255", UDP_PORT))
        try:
            data, addr = sock.recvfrom(1024)
            if data == PONG_MSG:
                esp_ip = addr[0]
                print(f"✅ ESP32-CAM found at {esp_ip}")
                sock.close()
                return esp_ip
        except socket.timeout:
            pass
        time.sleep(0.5)

    sock.close()
    print("❌ ESP32-CAM not found")
    return None

```

Observed Behaviour

The system operates stably in noisy environments, recovers predictably from network interruptions, and maintains correct identity separation across interactions. No such long-term learning or adaptation is observed. System behaviour remains dependent on prompt structure and external inference latency.

Comparison with Traditional Robotics
Traditional robotics systems rely on deterministic logic and predefined rules, while this system delegates high-level interpretation to a probabilistic model. Adaptation in traditional systems is manual, whereas this system adapts input handling dynamically. Safety in traditional systems is often compile-time, while this system enforces safety at runtime through validation and constraints.

Limitations
The LLM cannot reason about raw sensor data and depends entirely on text-based abstractions. Behaviour quality is sensitive to prompt design. Intelligence is externally hosted and subject to latency. No reinforcement learning or persistent memory adaptation is implemented.

Conclusion
This work demonstrates that a Large Language Model can be integrated into a real-time robotic loop only when surrounded by strict deterministic controls. The model does not learn, understand, or form internal representations of the environment. Its role is limited to probabilistic interpretation within carefully defined boundaries. The contribution of this project lies in system design and robustness engineering rather than claims of artificial intelligence.